Path 1: Pre Security (Legacy) > Module 4: Linux Fundamentals > Room 3: Linux Fundamentals Part 3

https://tryhackme.com/room/linuxfundamentalspart3

Linux Fundamentals Part 3:
Power-up your Linux skills and get hands-on with some common utilities that you are likely to use day-to-day!

>> Task 1: Introduction

Welcome to part three (and the finale) of the Linux Fundamentals module. So far, throughout the series, you have got hands-on with some fundamental concepts and used some important commands. This room is going to showcase some useful utilities and applications that you are likely to use day-to-day. You're also going to advance your Linux-fu skills by learning about automation, package management, and service/application logging.

>> Task 2: Deploy Your Linux Machine

Machine IP Address: 10.81.135.148
Username: tryhackme
password: tryhackme

ssh tryhackme@10.81.135.148

>> Task 3: Terminal Text Editors

- Introducing terminal text editors:

There are a few options that you can use, all with a variety of friendliness and utility. This task is going to introduce you to nano but also show you an alternative named **VIM** (which TryHackMe has a room dedicated to!)

Nano:
It is easy to get started with Nano! To create or edit a file using nano, we simply use nano filename -- replacing "filename" with the name of the file you wish to edit.

Tool: nano
Command: nano <filename>
Example: nano myfile

VIM:
VIM is a much more advanced text editor. Whilst you're not expected to know all advanced features, it's helpful to mention it for powering up your Linux skills.

Some of VIM's benefits, albeit taking a much longer time to become familiar with, includes:

- Customisable - you can modify the keyboard shortcuts to be of your choosing

- Syntax Highlighting - this is useful if you are writing or maintaining code, making it a popular choice for software developers

- VIM works on all terminals where nano may not be installed

- There are a lot of resources such as cheatsheets (https://vim.rtorr.com/), tutorials, and the sorts available to you use.

TryHackMe has a room (https://tryhackme.com/room/toolboxvim) showcasing VIM if you wish to learn more about this editor!

Q1) Edit "task3" located in "tryhackme"'s home directory using Nano. What is the flag?
Answer: THM{TEXT_EDITORS}

>> Task 4: General/Useful Utilities

- Downloading Files (Wget):

A pretty fundamental feature of computing is the ability to transfer files. For example, you may want to download a program, a script, or even a picture. Thankfully for us, there are multiple ways in which we can retrieve these files.

We're going to cover the use of wget . This command allows us to download files from the web via HTTP -- as if you were accessing the file in your browser. We simply need to provide the address of the resource that we wish to download. For example, if I wanted to download a file named "myfile.txt" onto my machine, assuming I knew the web address it -- it would look something like this:

wget https://assets.tryhackme.com/additional/linux-fundamentals/part3/myfile.txt

- Transferring Files From Your Host - SCP (SSH)

Secure copy, or SCP, is just that -- a means of securely copying files. Unlike the regular cp command, this command allows you to transfer files between two computers using the SSH protocol to provide both authentication and encryption.

Working on a model of SOURCE and DESTINATION, SCP allows you to:

- Copy files & directories from your current system to a remote system
- Copy files & directories from a remote system to your current system

Provided that we know usernames and passwords for a user on your current system and a user on the remote system. For example, let's copy an example file from our machine to a remote machine, which I have neatly laid out in the table below:

Copying Files from Our Local Machine to Remote Machine

Tool: scp
Syntax: scp <LocalMachineFile> <UsernameOfRemoteMachine>@<RemoteMachineIP>:<PathOfRemoteMachineToCopyFile>
Example: scp MyFile tryhackme@10.81.147.166:/home/tryhackme/

Variable	Value
The IP address of the remote system	192.168.1.30
User on the remote system	ubuntu
Name of the file on the local system	important.txt
Name that we wish to store the file as on the remote system	transferred.txt

With this information, let's craft our scp command (remembering that the format of SCP is just SOURCE and DESTINATION)

scp important.txt ubuntu@192.168.1.30:/home/ubuntu/transferred.txt

And now let's reverse this and layout the syntax for using scp to copy a file from a remote computer that we're not logged into

Copying Files from Remote Machine to Our Local Machine

Tool: scp
Syntax: scp <UsernameOfRemoteMachine>@<RemoteMachineIP>:<PathOfRemoteMachineFile> <GiveFileName>
Example: scp tryhackme@10.81.147.166:/home/tryhackme/MyFile.txt NewFile.txt
Example: scp tryhackme@10.81.147.166:/home/tryhackme/MyFile.txt MyFile.txt
Example: scp tryhackme@10.81.147.166:/home/tryhackme/MyFile NewFile

Variable	Value
IP address of the remote system	192.168.1.30
User on the remote system	ubuntu
Name of the file on the remote system	documents.txt
Name that we wish to store the file as on our system	notes.txt

scp ubuntu@192.168.1.30:/home/ubuntu/documents.txt notes.txt

- Serving Files From Your Host - WEB:

Ubuntu machines come pre-packaged with python3. Python helpfully provides a lightweight and easy-to-use module called "HTTPServer". This module turns your computer into a quick and easy web server that you can use to serve your own files, where they can then be downloaded by another computing using commands such as curl and wget.

Python3's "HTTPServer" will serve the files in the directory where you run the command, but this can be changed by providing options that can be found within the manual pages. Simply, all we need to do is run python3 -m http.server in the terminal to start the module! In the snippet below, we are serving from a directory called "webserver", which has a single named "file".

Using Python to start a web server:
python3 -m http.server (Terminal 1)

Now, let's use wget to download the file using the 10.80.144.54(Server Machine IP) address and the name of the file. Remember, because the python3 server is running port 8000, you will need to specify this within your wget command. For example:

An example wget command of a web server running on port 8000:

tryhackme@mymachine:~# wget http://10.80.144.54:8000/myfile

Note, you will need to open a new terminal to use wget and leave the one that you have started the Python3 web server in. This is because, once you start the Python3 web server, it will run in that terminal until you cancel it.

Downloading a file from our webserver using wget
tryhackme@linux3:/tmp# wget http://10.80.144.54:8000/file (Terminal 2)

Syntax: wget http://<Python3 Server Machine IP>:<Python3WebServerRunningPort>/<File/FolderName - That Exist/Stored On Python3 WebServer>
Example: wget http://10.80.144.54:8000/myfile
Example: wget http://10.80.144.54:8000/myfolder

You Can Access/See Files/Folders Using Your Browser by Entering: http://<Python3 Server Machine IP>:<Python3WebServerRunningPort>
Example: http://10.80.144.54:8000/

One flaw with this module is that you have no way of indexing, so you must know the exact name and location of the file that you wish to use. This is why I prefer to use Updog (https://github.com/sc0tfree/updog). What's Updog? A more advanced yet lightweight webserver. But for now, let's stick to using Python's "HTTP Server".

Documentation on Python3's "HTTPServer" module. (https://docs.python.org/3/library/http.server.html)

Q1) Download the file http://10.80.144.54:8000/.flag.txt onto the TryHackMe AttackBox. Remember, you will need to do this in a new terminal.
What are the contents?
Answer: THM{WGET_WEBSERVER}

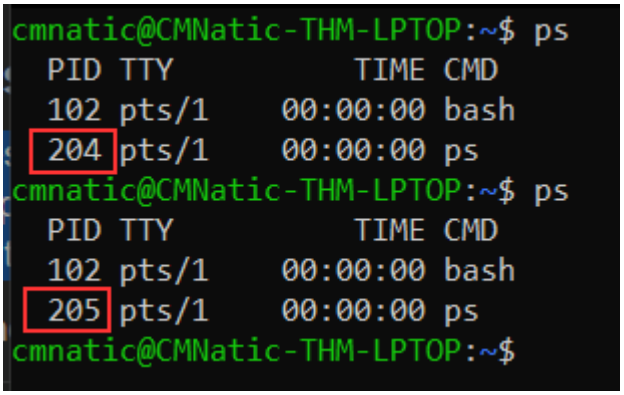
>> Task 5: Processes 101

Processes are the programs that are running on your machine. They are managed by the kernel, where each process will have an ID associated with it, also known as its PID. The PID increments for the order In which the process starts. I.e. the 60th process will have a PID of 60.

- Viewing Processes

We can use the friendly ps command to provide a list of the running processes as our user's session and some additional information such as its status code, the session that is running it, how much usage time of the CPU it is using, and the name of the actual program or command that is being executed.

Command: ps



To see the processes run by other users and those that don't run from a session (i.e. system processes), we need to provide **aux** to the ps command like so: ps aux

Main Command: ps
Sub Command: ps aux

Another very useful command is the top command; top gives you real-time statistics about the processes running on your system instead of a one-time view. These statistics will refresh every 10 seconds, but will also refresh when you use the arrow keys to browse the various rows. Another great command to gain insight into your system is via the top command

Command: top

```
top - 22:36:17 up 1 day,  6:32,  0 users,  load average: 0.00, 0.00, 0.00
Tasks:   5 total,   1 running,   4 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.8 sy,  0.0 ni, 99.2 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  12630.9 total,  12206.5 free,     83.6 used,    340.9 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.  12306.1 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root      20   0     892    580    516 S   0.0   0.0   0:00.93 init
  100 root      20   0     892     84     20 S   0.0   0.0   0:00.00 init
  101 root      20   0     892     84     20 S   0.0   0.0   0:00.07 init
  102 cmnatic   20   0   10032   4988   3272 S   0.0   0.0   0:00.08 bash
  209 cmnatic   20   0   10872   3704   3188 R   0.0   0.0   0:00.00 top
```

- Managing Processes:

You can send signals that terminate processes; there are a variety of types of signals that correlate to exactly how "cleanly" the process is dealt with by the kernel. To kill a command, we can use the appropriately named kill command and the associated PID that we wish to kill. i.e., to kill PID 1337, we'd use kill 1337.

Command: kill
Syntax: kill <PID>
Example: kill 1219

Below are some of the signals that we can send to a process when it is killed:

SIGTERM - Kill the process, but allow it to do some cleanup tasks beforehand
SIGKILL - Kill the process - doesn't do any cleanup after the fact
SIGSTOP - Stop/suspend a process

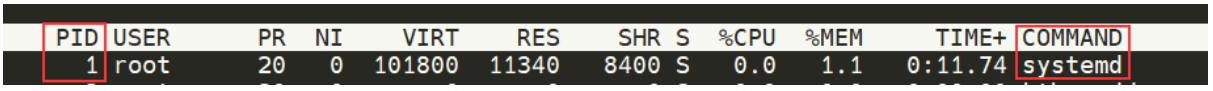
- How do Processes Start?

Let's start off by talking about namespaces. The Operating System (OS) uses namespaces to ultimately split up the resources available on the computer to (such as CPU, RAM and priority) processes. Think of it as splitting your computer up into slices -- similar to a cake. Processes within that slice will have access to a certain amount of computing power, however, it will be a small portion of what is actually available to every process overall.

Namespaces are great for security as it is a way of isolating processes from another -- only those that are in the same namespace will be able to see each other.

We previously talked about how PID works, and this is where it comes into play. The process with an ID of 0 is a process that is started when the system boots. This process is the system's init on Ubuntu, such as systemd, which is used to provide a way of managing a user's processes and sits in between the operating system and the user.

For example, once a system boots and it initialises, systemd is one of the first processes that are started. Any program or piece of software that we want to start will start as what's known as a child process of systemd. This means that it is controlled by systemd, but will run as its own process (although sharing the resources from systemd) to make it easier for us to identify and the likes.



- Getting Processes/Services to Start on Boot:

Some applications can be started on the boot of the system that we own. For example, web servers, database servers or file transfer servers. This software is often critical and is often told to start during the boot-up of the system by administrators.

In this example, we're going to be telling the apache web server to be starting apache manually and then telling the system to launch apache2 on boot.

Enter the use of systemctl -- this command allows us to interact with the systemd process/daemon. Continuing on with our example, systemctl is an easy to use command that takes the following formatting: systemctl [option] [service]

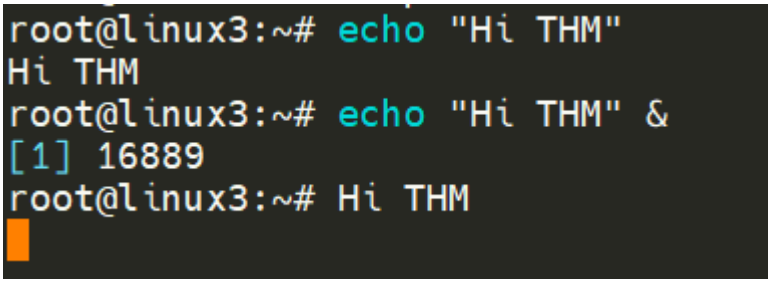
For example, to tell apache to start up, we'll use systemctl start apache2. Seems simple enough, right? Same with if we wanted to stop apache, we'd just replace the [option] with stop (instead of start like we provided)

We can do four options with systemctl:

Start
Stop
Enable
Disable

- An Introduction to Backgrounding and Foregrounding in Linux

Processes can run in two states: In the background and in the foreground. For example, commands that you run in your terminal such as "echo" or things of that sort will run in the foreground of your terminal as it is the only command provided that hasn't been told to run in the background. "Echo" is a great example as the output of echo will return to you in the foreground, but wouldn't in the background.



Here we're running echo "Hi THM" , where we expect the output to be returned to us like it is at the start. But after adding the & operator to the command, we're instead just given the ID of the echo process rather than the actual output -- as it is running in the background.

This is great for commands such as copying files because it means that we can run the command in the background and continue on with whatever further commands we wish to execute (without having to wait for the file copy to finish first).

We can do the exact same when executing things like scripts -- rather than relying on the & operator, we can use Ctrl + Z on our keyboard to background a process. It is also an effective way of "pausing" the execution of a script or command.

```
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
This will keep on looping until I stop it!
T^Z
[1]+  Stopped                 ./background.sh
root@linux3:/var/opt#
```

Example:
Run: ping <ip/domain> Then ctrl+Z

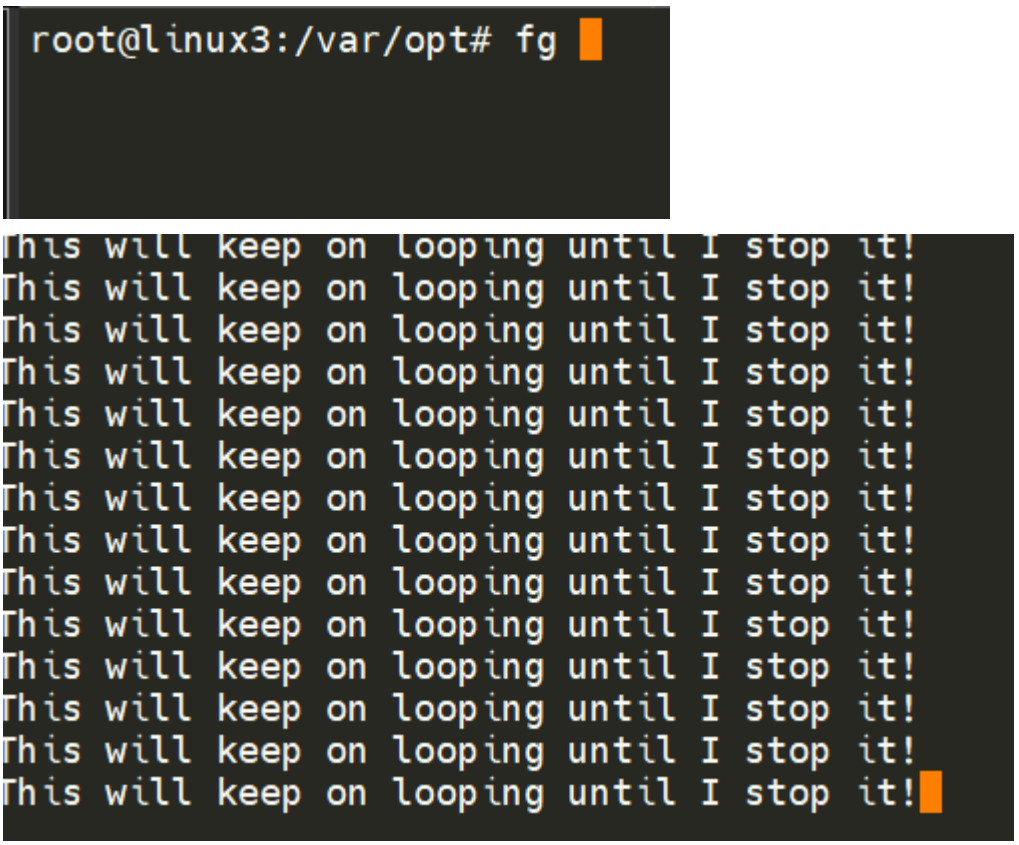
This script will keep on repeating "This will keep on looping until I stop!" until I stop or suspend the process. By using Ctrl + Z (as indicated by **T^Z**). Now our terminal is no longer filled up with messages -- until we foreground it.

- Foregrounding a process:

Now that we have a process running in the background, for example, our script "background.sh" which can be confirmed by using the ps aux command, we can back-pedal and bring this process back to the foreground to interact with.

```
root       19802  0.9  0.3   8616  3108 pts/1    T    15:37   0:01 /bin/bash ./background.sh
root       20995  0.0  0.0      0     0 ?        I    15:40   0:00 [kworker/u30:1-events_unbound]
ubuntu     21007  0.0  0.0   7228   592 ?        S    15:40   0:00 sleep 1
root       21008  0.0  0.3  10616  3452 pts/1    R+   15:40   0:00 ps aux
root@linux3:/var/opt#
```

With our process backgrounded using either Ctrl + Z or the & operator, we can use fg to bring this back to focus like below, where we can see the fg command is being used to bring the background process back into use on the terminal, where the output of the script is now returned to us.



Command to check running processes: ps aux
Command to bring back background process to foreground: fg

Q1) If we were to launch a process where the previous ID was "300", what would the ID of this new process be?
Answer: 301

Q2) If we wanted to **cleanly** kill a process, what signal would we send it?
Answer: SIGTERM

Q3) Locate the process that is running on the deployed instance (10.82.179.180). What flag is given?
Answer: THM{PROCESSES}
Hint: ps aux

Q4) What command would we use to stop the service "myservice"?
Answer: systemctl stop myservice

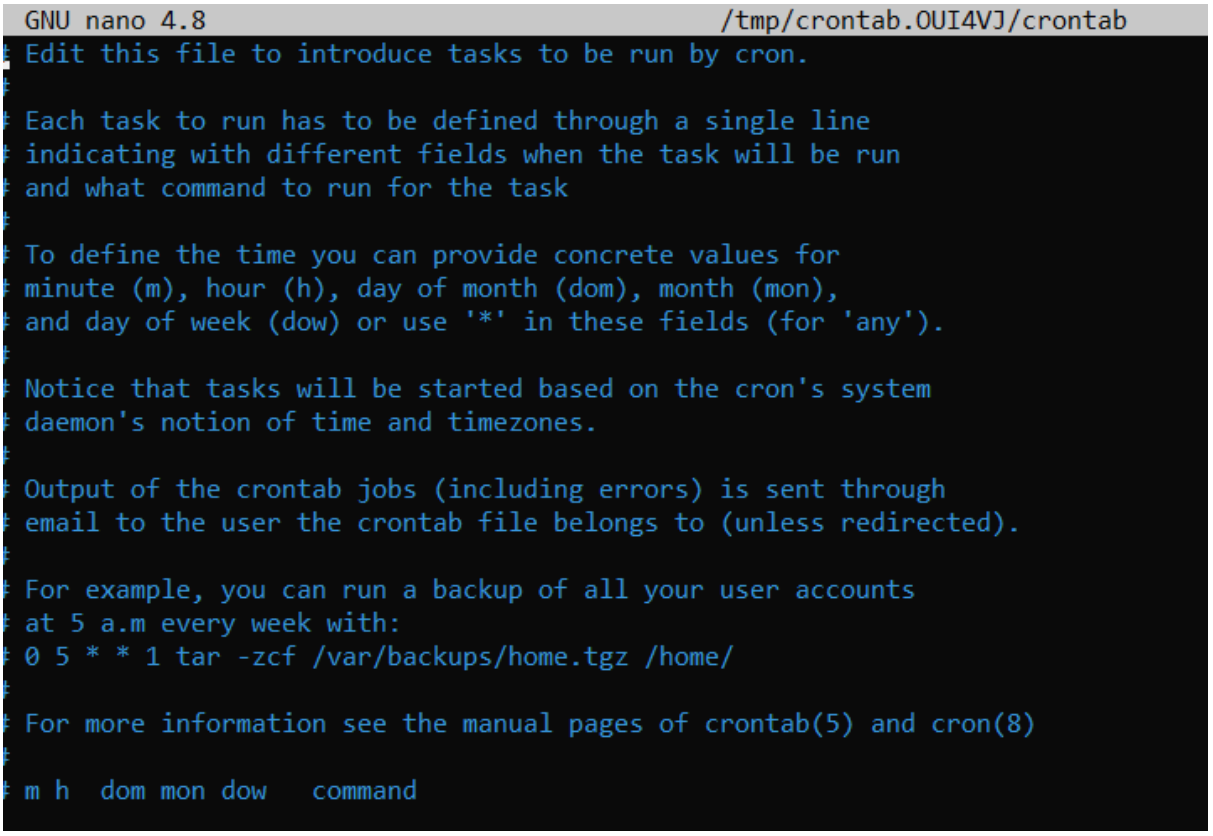
Q5) What command would we use to start the same service on the boot-up of the system?
Answer: systemctl enable myservice

Q6) What command would we use to bring a previously backgrounded process back to the foreground?
Answer: fg

>> Task 6: Maintaining Your System: Automation

Users may want to schedule a certain action or task to take place after the system has booted. Take, for example, running commands, backing up files, or launching your favourite programs on, such as Spotify or Google Chrome.

We're going to be talking about the **cron** process, but more specifically, how we can interact with it via the use of **crontabs**. Crontab is one of the processes that is started during boot, which is responsible for facilitating and managing cron jobs.



A crontab is simply a special file with formatting that is recognised by the **cron** process to execute each line step-by-step. Crontabs require 6 specific values:

Value	**Description**
MIN	What minute to execute at
HOUR	What hour to execute at
DOM	What day of the month to execute at
MON	What month of the year to execute at
DOW	What day of the week to execute at

CMD	The actual command that will be executed

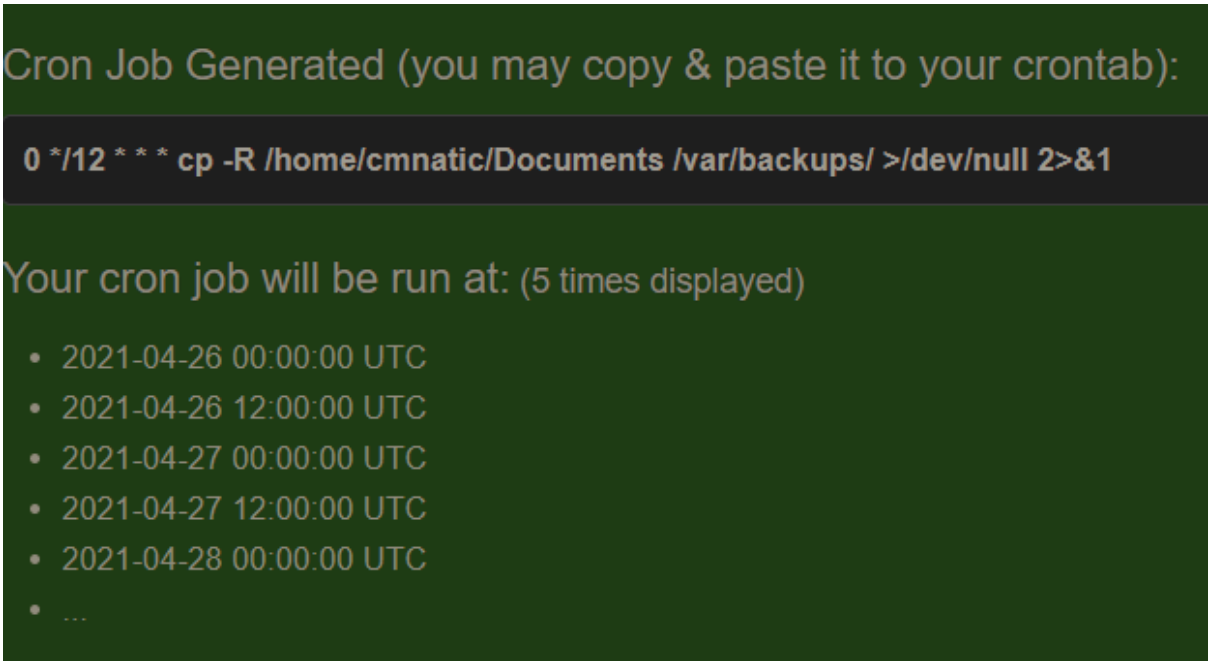
Let's use the example of backing up files. You may wish to backup "cmnatic"'s "Documents" every 12 hours. We would use the following formatting:

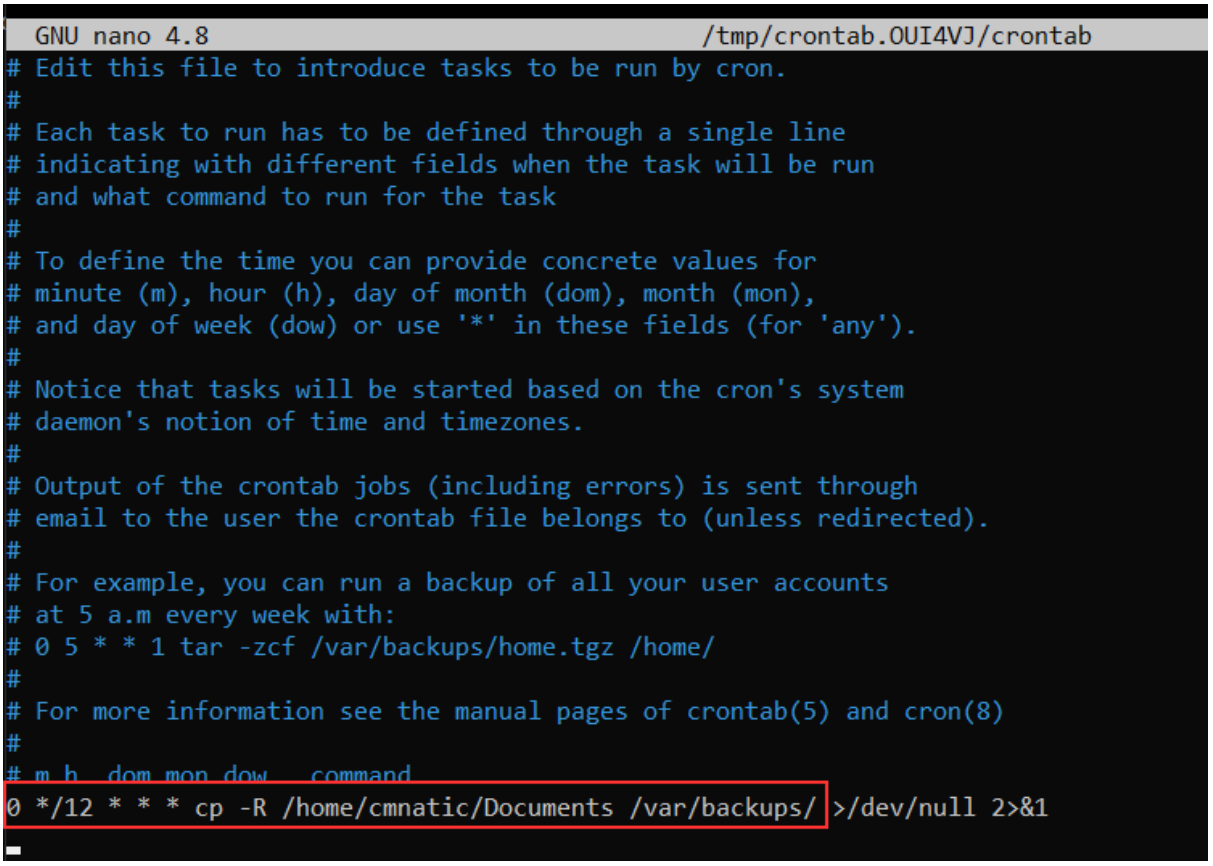
0 */12 * * * cp -R /home/cmnatic/Documents /var/backups/

An interesting feature of crontabs is that these also support the wildcard or asterisk (*). If we do not wish to provide a value for that specific field, i.e. we don't care what month, day, or year it is executed -- only that it is executed every 12 hours, we simply just place an asterisk.

This can be confusing to begin with, which is why there are some great resources such as the online "Crontab Generator" (https://crontab-generator.org/) that allows you to use a friendly application to generate your formatting for you! As well as the site "Cron Guru"! (https://crontab.guru/)

Crontabs can be edited by using crontab -e, where you can select an editor (such as Nano) to edit your crontab.





Q1) Ensure you are connected to the deployed instance and look at the running crontabs.
Answer: crontab -e

Q2) When will the crontab on the deployed instance (10.82.179.180) run?
Answer: @reboot

>> Task 7: Maintaining Your System: Package Management

- Introducing Packages & Software Repos

When developers wish to submit software to the community, they will submit it to an "apt" repository. If approved, their programs and tools will be released into the wild. Two of the most redeeming features of Linux shine to light here: User accessibility and the merit of open source tools.

When using the ls command on a Ubuntu 20.04 Linux machine, these files serve as the gateway/registry.

```
ubuntu@ip-10-10-29-121:/etc/apt$ ls
apt.conf.d  auth.conf.d  preferences.d  sources.list  sources.list.d  trusted.gpg.d
ubuntu@ip-10-10-29-121:/etc/apt$
```

```
  GNU nano 2.9.3                          sources.list


## Uncomment the following two lines to add software from Canonical's
## 'partner' repository.
## This software is not part of Ubuntu, but is offered by Canonical and the
## respective vendors as a service to Ubuntu users.
# deb http://archive.canonical.com/ubuntu bionic partner
# deb-src http://archive.canonical.com/ubuntu bionic partner

deb http://security.ubuntu.com/ubuntu bionic-security main restricted
# deb-src http://security.ubuntu.com/ubuntu bionic-security main restricted
deb http://security.ubuntu.com/ubuntu bionic-security universe
# deb-src http://security.ubuntu.com/ubuntu bionic-security universe
deb http://security.ubuntu.com/ubuntu bionic-security multiverse
# deb-src http://security.ubuntu.com/ubuntu bionic-security multiverse
```

Whilst Operating System vendors will maintain their own repositories, you can also add community repositories to your list! This allows you to extend the capabilities of your OS. Additional repositories can be added by using the add-apt-repository command or by listing another provider! For example, some vendors will have a repository that is closer to their geographical location.

- Managing Your Repositories (Adding and Removing)

Normally we use the apt command to install software onto our Ubuntu system. The apt command is a part of the package management software also named apt. Apt contains a whole suite of tools that allows us to manage the packages and sources of our software, and to install or remove software at the same time.

One method of adding repositories is to use the add-apt-repository command we illustrated above, but we're going to walk through adding and removing a repository manually. Whilst you can install software through the use of package installers such as dpkg, the benefits of apt means that whenever we update our system -- the repository that contains the pieces of software that we add also gets checked for updates.

In this example, we're going to add the text editor Sublime Text to our Ubuntu machine as a repository as it is not a part of the default Ubuntu repositories. When adding software, the integrity of what we download is guaranteed by the use of what is called GPG (Gnu Privacy Guard) keys. These keys are essentially a safety check from the developers saying, "here's our software". If the keys do not match up to what your system trusts and what the developers used, then the software will not be downloaded.

So, to start, we need to add the GPG key for the developers of Sublime Text 3. (Note that TryHackMe instances do not have internet access and so we're not expecting you to add this to the machine that you deploy, as it would fail.)

1. Let's download the GPG key and use apt-key to trust it: wget -qO - https://download.sublimetext.com/sublimehq-pub.gpg | sudo apt-key add -

2. Now that we have added this key to our trusted list, we can now add Sublime Text 3's repository to our apt sources list. A good practice is to have a separate file for every different community/3rd party repository that we add.

2.1. Let's create a file named **sublime-text.list in /etc/apt/sources.list.d** and enter the repository information like so:

```
root@linux3:/etc/apt/sources.list.d# touch sublime-text.list
root@linux3:/etc/apt/sources.list.d# ls
sublime-text.list
root@linux3:/etc/apt/sources.list.d#
```

2.2. And now use Nano or a text editor of your choice to add & save the Sublime Text 3 repository into this newly created file:

```
  GNU nano 4.8                                                    s
deb https://download.sublimetext.com/ apt/stable/
```

2.3. After we have added this entry, we need to update apt to recognise this new entry -- this is done using the apt update command

2.4. Once successfully updated, we can now proceed to install the software that we have trusted and added to apt using apt install sublime-text

Removing packages is as easy as reversing. This process is done by using the add-apt-repository --remove ppa:PPA_Name/ppa command or by manually deleting the file that we previously added to. Once removed, we can just use apt remove [software-name-here] i.e. apt remove sublime-text

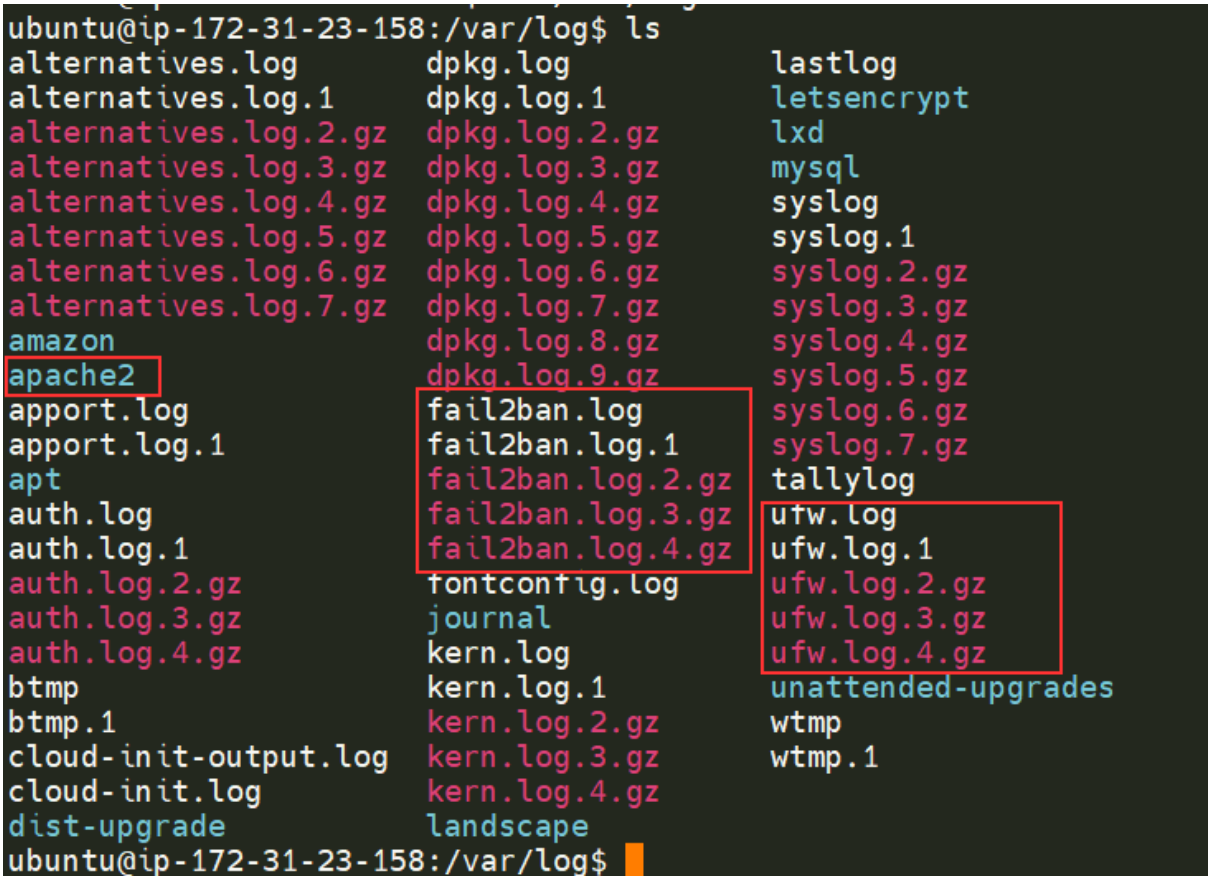
Since TryHackMe instances do not have an internet connection...this task only requires you to read through the material.

>> Task 8: Maintaining Your System: Logs

We briefly touched upon log files and where they can be found in Linux Fundamentals Part 1. However, let's quickly recap. Located in the /var/log directory, these files and folders contain logging information for applications and services running on your system. The Operating System (OS) has become pretty good at automatically managing these logs in a process that is known as "rotating".

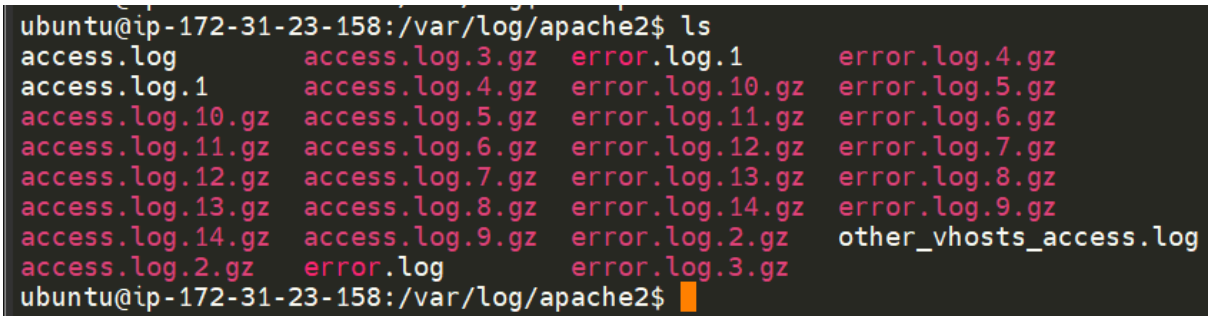
I have highlighted some logs from three services running on a Ubuntu machine:

- An Apache2 web server
- Logs for the fail2ban service, which is used to monitor attempted brute forces, for example
- The UFW service which is used as a firewall



These services and logs are a great way in monitoring the health of your system and protecting it. Not only that, but the logs for services such as a web server contain information about every single request - allowing developers or administrators to diagnose performance issues or investigate an intruder's activity. For example, the two types of log files below that are of interest:

- access log
- error log



There are, of course, logs that store information about how the OS is running itself and actions that are performed by users, such as authentication attempts.

Q1) Look for the apache2 logs on the deployable Linux machine.
Answer: Located in /var/log/apache2

Q2) What is the IP address of the user who visited the site?
Answer: 10.9.232.111

Q3) What file did they access?
Answer: catsanddogs.jpg

>> Task 9: Conclusions & Summaries

Welcome to the end of the Linux Fundamentals module. Your familiarity with Linux will improve as you get to interact with it over time. Linux has the potential to do very powerful things with relative ease (as you have hopefully discovered throughout this module)

To recap, this room introduced you to the following topics:

- Using terminal text editors

- General utilities such as downloading and serving contents using a python webserver

- A look into processes

- Maintaining & automating your system by the use of crontabs, package management, and reviewing logs

Continue your learning in some other TryHackMe rooms that are dedicated to Linux tools or utilities:

Bash Scripting - https://tryhackme.com/room/bashscripting

Regular Expressions - https://tryhackme.com/room/catregex